

Computer Organization and Architecture

Exhaustive Unit-1 Question Bank Solutions Manual (Lengthy & Detailed Explanations)

S.Y. B.Tech (Sem-III) | Academic Year: 2026-27

QUESTION 1

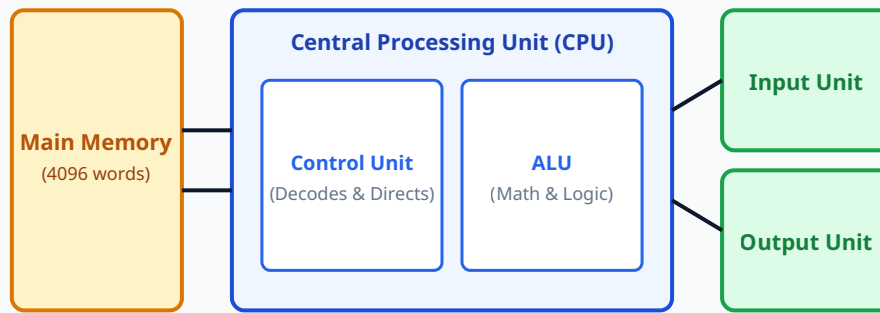
Draw the general structure of IAS computer and explain in detail.

The Institute for Advanced Study (IAS) computer, developed under the leadership of John von Neumann at Princeton University in the late 1940s and finalized around 1952, represents the foundational architectural blueprint for almost all modern digital electronic computers. Prior to the IAS machine, computers were largely configured for specific calculations with hardwired control sequences. The IAS computer popularized the *stored-program concept*.

| Detailed Architectural Components

The hardware organization of the IAS computer comprises four primary subsystems:

1. **Main Memory Unit:** The storage subsystem responsible for holding both instructions and data operands. In the original IAS design, memory consisted of 40,960 words, where each word was exactly 40 bits long. Each instruction comprised a 8-bit opcode and a 12-bit address field, often packing two instructions into a single 40-bit word.
2. **Arithmetic and Logic Unit (ALU):** The computational heart of the processor. It performs fundamental binary arithmetic (addition, subtraction, multiplication, division) and logical comparisons (AND, OR, NOT, shifts).
3. **Control Unit (CU):** The operational brain that orchestrates system timing and data flow. It fetches instructions from memory, interprets their opcodes, and emits specific control signals to direct registers, the ALU, and buses.
4. **Input/Output (I/O) Equipment:** Mechanisms that interface external operators, punched cards, magnetic tapes, and display consoles with the internal processing core.



General Structure of IAS Computer (Stored-Program Architecture)

Internal Register Set of the IAS Computer

- **Program Counter (PC):** Contains the 12-bit address of the next instruction pair to be fetched from memory.
- **Accumulator (AC) and Multiplier-Quotient (MQ):** Temporary working registers used to hold operands before arithmetic operations and store results generated by the ALU.
- **Memory Address Register (MAR):** Specifies the 12-bit address of the memory location currently being accessed for read or write operations.
- **Memory Buffer Register (MBR):** Contains the 40-bit data word either waiting to be written to memory or freshly read from memory.
- **Instruction Register (IR):** Holds the 8-bit opcode portion of the active instruction being executed.

QUESTION 2

Explain basic instruction cycle in detail.

The execution of a program by a CPU involves processing a sequence of instructions stored in primary memory. Each individual instruction is processed through a repeating sequence known as the **Instruction Cycle** (or Fetch-Decode-Execute cycle). In complex processors, this cycle incorporates sub-phases such as indirect address calculation and interrupt processing.

Phases of the Instruction Cycle

1. Fetch Phase:

At the beginning of every cycle, the processor needs to retrieve the next instruction from memory.

- The content of the **Program Counter (PC)** (holding the address of the upcoming instruction) is loaded into the **Memory Address Register (MAR)**.
- The Control Unit issues a read command over the control bus.
- The addressed memory word travels over the data bus into the **Memory Buffer Register (MBR)** and is subsequently routed to the **Instruction Register (IR)**.

- Finally, the PC is automatically incremented to point to the next instruction in sequence.

2. Decode Phase:

The Control Unit analyzes the bit pattern residing in the Instruction Register. It decodes the opcode to determine what operation needs to be executed, which registers are involved, and whether memory or I/O operands must be accessed.

3. Execute Phase:

The Control Unit activates the corresponding internal data paths and ALU control lines to accomplish the instruction's objective. Execution types include:

- *Processor-to-Memory*: Moving data between CPU registers and primary memory.
- *Processor-to-I/O*: Interfacing with peripheral input/output controllers.
- *Data Processing*: Performing arithmetic or logical computations via the ALU.
- *Control/Branch*: Altering the execution sequence by loading a new value into the PC (e.g., conditional or unconditional jumps).

Note on State Transitions: Between the Fetch and Execute phases, if an instruction requires indirect addressing, an *Indirect Cycle* evaluates effective memory addresses. Following execution, the processor checks for pending hardware interrupts before returning to the fetch stage.

QUESTION 3

What is Computer Organization? Explain the architecture structure of Computer Organization in detail.

Computer Organization refers to the operational units and their electrical/logical interconnections that realize the abstract architectural specifications of a computer system. While *computer architecture* deals with attributes visible to a programmer (such as instruction sets, data types, and addressing modes), computer organization deals with how hardware components (transistors, logic gates, buses, memory controllers, and peripheral interfaces) are structured and controlled to execute those architectural requirements.

Hierarchical Structural Levels in Computer Organization

Computer organization can be studied across multiple structural levels:

- **Top Level (System Level):** Encompasses the major subsystems—the Central Processing Unit (CPU), Main Memory, and Input/Output devices—interconnected via a shared system bus.
- **CPU Level (Processor Level):** Examines internal processor composition, including the Control Unit, Arithmetic Logic Unit (ALU), internal registers, and internal high-speed data paths.
- **Register Level:** Focuses on storage registers, multiplexers, decoders, and buses constructed from flip-flops and combinational logic blocks.

- **Digital Logic Level:** The foundational level consisting of logic gates (AND, OR, NOT, NAND, NOR, XOR) built from semiconductor transistors.

QUESTION 4 & 10

Write a short note on Interconnection Structure / Bus Interconnection.

An **Interconnection Structure** is a collection of communication pathways—commonly referred to as a **Bus**—that links the fundamental functional components of a computer system (CPU, Memory, and I/O modules). Without an effective interconnection structure, individual hardware units cannot exchange instructions, operands, and control status.

Classification of Bus Lines in a System Bus

A standard system bus consists of multiple parallel electrical wires categorized into three functional groups:

1. **Data Bus:** Transports actual data instructions and numerical operands between the CPU, memory, and peripheral controllers. The data bus is *bidirectional*, and its bit-width (e.g., 32-bit, 64-bit) directly dictates the word size transfer capacity per clock cycle.
2. **Address Bus:** Carries memory addresses or I/O port identifiers to designate the source or destination of a data transfer. The address bus is *unidirectional* (from CPU to memory/I/O), and its width determines the maximum addressable memory space (2^N locations).
3. **Control Bus:** Manages transmission coordination by carrying command signals and timing cues (such as Memory Read, Memory Write, I/O Read, I/O Write, Bus Request, Bus Grant, and Interrupt Acknowledge) across system components.

QUESTION 5 & 11

Write a note on the brief history of computers and explain different generations in detail.

The evolution of digital computing is categorized into distinct generations defined by rapid breakthroughs in electronic component technology, memory density, and software ecosystems.

Generation	Approx. Period	Core Hardware Technology	Architectural & Software Characteristics
First Generation	1940–1956	Vacuum Tubes	Massive physical size, high power consumption, punched card input, machine and assembly language. (ENIAC, UNIVAC).
Second Generation	1956–1963	Transistors	

			Replaced vacuum tubes, reliable, smaller footprint, magnetic core memory, high-level languages (FORTRAN, COBOL).
Third Generation	1964–1971	Integrated Circuits (ICs)	Silicon semiconductor chips hosting multiple transistors, introduction of operating systems, keyboards, and monitors.
Fourth Generation	1971–Present	Microprocessors (VLSI)	Millions of transistors on a single chip (Very Large Scale Integration), birth of personal computers, workstations, and internet connectivity.
Fifth Generation	Present & Beyond	AI & ULSI (Ultra-Large Scale Integration)	Massive parallel processing, neural networks, expert systems, advanced artificial intelligence, and quantum computing paradigms.

QUESTION 6

Draw and explain the von Neumann architecture.

Proposed by mathematician John von Neumann in his 1945 report on the EDVAC, the **von Neumann Architecture** established the stored-program computer model. Prior architectures required manual rewiring or plugboard reprogramming for each computation. Von Neumann proposed storing both data and program instructions together in the same read-write primary memory unit.

Core Principles of von Neumann Architecture

- **Single Memory Space:** Both data and instructions share the same physical memory addresses and data paths.
- **Sequential Execution:** Instructions are fetched and executed sequentially under the governance of a Program Counter, unless altered by a jump/branch instruction.
- **Centralized Processing Unit:** Contains an ALU for computations and a Control Unit for sequencing operations.

The von Neumann Bottleneck: Because data and instruction fetches share a single memory bus, the CPU cannot read an instruction and read/write data simultaneously. This creates a fundamental throughput limitation known as the von Neumann bottleneck, mitigated in modern processors by separate L1 instruction and data caches (Harvard architecture principle implemented internally).

QUESTION 7

List the features of second, third, and fourth generation of computers.

A comparative look at the technical breakthroughs across computer generations highlights exponential advances in computing power:

- **Second Generation (Transistors):**

- Utilization of solid-state semiconductor transistors instead of fragile, heat-producing vacuum tubes.
- Substantial reduction in physical size, heat generation, and electrical power requirements.
- Calculation speeds advanced from milliseconds into microseconds.
- Emergence of early commercial compilers and high-level programming languages (FORTRAN, ALGOL, COBOL).

- **Third Generation (Integrated Circuits - ICs):**

- Invention of planar IC technology, etching multiple transistors, resistors, and capacitors onto single silicon microchips.
- Introduction of commercial operating systems enabling batch processing and multi-programming.
- Standardized input/output peripherals including interactive keyboards and cathode-ray tube (CRT) monitors.

- **Fourth Generation (Microprocessors - VLSI):**

- Integration of entire Central Processing Units onto a single Very Large Scale Integration (VLSI) silicon chip.
- Catalyzed the microcomputer revolution, making affordable personal computers accessible to businesses and households.
- Introduction of graphical user interfaces (GUIs), semiconductor solid-state memory chips, and global networking protocols.

QUESTION 8

Explain the techniques used to increase processor speed.

As computational demands escalated, relying solely on semiconductor clock frequency scaling (Dennard scaling limits) became insufficient. Modern architectures employ several hardware and organizational techniques to maximize processor speed and instruction throughput:

1. **Instruction Pipelining:** Overlapping the execution stages of sequential instructions. While instruction 1 is executing in the ALU, instruction 2 is being decoded, and instruction 3 is being fetched, maximizing hardware utilization.
2. **Cache Memory Hierarchy:** Integrating high-speed static RAM (SRAM) levels (L1, L2, L3 caches) directly on or near the CPU die to maintain frequently accessed data and instructions closer to the execution units, bypassing slower main memory (DRAM) latency.

3. **Superscalar Architecture:** Equipping the processor core with multiple parallel execution units (multiple ALUs and floating-point units), enabling the CPU to dispatch and execute more than one instruction per clock cycle.
4. **Branch Prediction and Speculative Execution:** Utilizing hardware algorithms to predict conditional branch outcomes before they are resolved, allowing the processor to speculatively execute instructions ahead of time.
5. **Multi-Core and Multithreading:** Placing multiple complete CPU cores onto a single silicon chip (multi-core) alongside Simultaneous Multithreading (SMT/Hyper-Threading) to execute parallel threads concurrently.

QUESTION 9

Draw and explain the block diagram of a simple computer with five functional units.

Every digital computer system, regardless of scale, is structured around five core functional units that work in tandem to accept input, store information, perform computations, coordinate control, and present output.

- **1. Input Unit:** Accepts raw data, user commands, and programs from external input devices (keyboards, mice, scanners, sensors) and converts them into binary machine-understandable signals.
- **2. Memory Unit (Storage):** Retains program instructions during execution and stores data operands. Subdivided into primary storage (RAM/ROM) for active access and secondary storage (SSDs/HDDs) for persistent retention.
- **3. Arithmetic Logic Unit (ALU):** Executes all mathematical computations (addition, subtraction, multiplication, division) and logical decisions (AND, OR, comparisons).
- **4. Control Unit (CU):** Acts as the system supervisor, fetching instructions, interpreting opcodes, and synchronizing data flow across all hardware modules. (Note: ALU + CU constitute the CPU).
- **5. Output Unit:** Translates processed binary results from the computer into human-readable or machine-actuated formats via monitors, printers, or speakers.

QUESTION 12

Draw and explain the multiple bus structures.

A single bus architecture connects the CPU, memory, and all I/O devices to one shared communication highway. While cost-effective, it creates severe performance contention because only one master device can transmit data across the bus at any given instant.

Multiple Bus Structures

To eliminate this bottleneck, modern computers utilize **Multiple Bus Structures**, deploying two or more specialized buses interconnected through bus bridges, memory controllers, and I/O hubs:

- **Dedicated Memory Bus:** Directly links the CPU to high-speed cache and main memory modules, allowing unhindered instruction and data fetches.
- **System and Expansion Buses (PCIe/USB):** Connect peripheral controllers, graphics cards, and storage drives on separate high-bandwidth channels.

Advantage: Enables concurrent operations—for instance, a Direct Memory Access (DMA) controller can transfer data directly between an I/O device and RAM on an expansion bus while the CPU simultaneously executes instructions using cache memory without bus conflict.

QUESTION 13

Explain the different types of Buses.

Buses are communication pathways that transfer information between computer hardware components. They are classified based on their functional location within the hierarchy:

1. **Internal Processor Bus:** Short pathways connecting internal CPU registers, the ALU, and internal control multiplexers within the processor chip.
2. **System Bus (Front Side Bus):** The primary motherboard bus connecting the CPU, main memory (RAM), and cache subsystems. Composed of data, address, and control lines.
3. **Expansion / I/O Bus:** Connects peripheral expansion slots (PCI Express, USB controllers, SATA storage interfaces) to the core system architecture, accommodating devices with varying data transfer speeds.

QUESTION 14

Write a short note on Traditional Bus Structure and High Performance Bus Structure.

Traditional Bus Structure: Early computer architectures (such as ISA or early PCI bus standards) relied on a single unified bus hierarchy connecting all system components—CPU, memory, graphics cards, and slow I/O devices—to a common pathway. This caused high-speed devices and low-speed devices to contend for the exact same bandwidth, severely limiting overall throughput.

High-Performance Bus Structure: Modern architectures (such as Intel Hub Architecture or AMD Infinity Fabric) decouple high-speed subsystems from slower ones. A dedicated high-speed *Memory Bus* links the CPU directly to cache and RAM, while a high-bandwidth graphics bus (PCIe) connects the GPU. Slower I/O peripherals communicate through an auxiliary southbridge or I/O platform controller hub, ensuring optimal bandwidth allocation across all devices.

QUESTION 15

Draw and explain the architecture of 8086 microprocessor.

The Intel 8086 is a 16-bit, 40-pin microprocessor introduced in 1978. Its internal organization is split into two independent functional units that operate concurrently using pipelining:

- **1. Bus Interface Unit (BIU):**

Handles all external bus operations and data transfers. Responsibilities include fetching instructions from memory, reading/writing operands, and computing physical memory addresses. It contains:

- Segment Registers (**CS, DS, SS, ES**) for memory segmentation.
- Instruction Pointer (**IP**) pointing to the next instruction offset.
- A 6-byte **Instruction Queue** that prefetches instructions ahead of time to minimize CPU idle wait states.

- **2. Execution Unit (EU):**

Responsible for decoding and executing instructions supplied by the BIU queue. It contains:

- General Purpose Registers (**AX, BX, CX, DX**).
- Pointer and Index Registers (**SP, BP, SI, DI**).
- The Arithmetic Logic Unit (ALU) and a 16-bit Flag Register reflecting arithmetic status and control states.

QUESTION 16, 17 & 18

Explain 8086 Addressing Modes with Detailed Examples & Instruction Identification.

Addressing modes refer to the architectural method by which an instruction specifies the operand's location. The 8086 microprocessor supports versatile addressing modes:

1. Immediate Addressing Mode:

The operand is specified explicitly within the instruction itself.

Example: **MOV AX, 2550H** (Loads the 16-bit constant value **2550H** directly into register **AX**).

2. Register Indirect Addressing Mode:

The effective memory address of the operand is held inside a base or index register (**BX, BP, SI, DI**).

Example for Q17: **MOV [SI], 2050H** — Here, **[SI]** employs **Register Indirect Addressing**, meaning the immediate data **2050H** is stored into the memory location whose offset address is specified by the contents of the **SI** register.

3. Indexed / Direct Addressing Mode:

The effective address is derived from a direct offset specified in the instruction.

Example for Q17: **MOV BX, [1000H]** — This instruction utilizes **Direct Memory Addressing**, where the offset address **1000H** is explicitly provided within the instruction code, and the 16-bit word stored at that memory location is fetched and loaded into register **BX**.